
Glossaire

A

a11y *accessibility*, voir **accessibilité**.

abstraction généralisation d'une idée ou d'un concept. C'est une simplification ne gardant que les caractéristiques essentielles des éléments considérés, et s'appliquant à différents exemples concrets. Elle permet de manipuler un seul concept plutôt que différents cas particuliers, et masque les détails sous un concept commun. C'est une forme de généralisation des concepts.

accessibilité conception d'un système, en particulier interactif, de sorte à pouvoir être facilement utilisé par des personnes en situation de handicap. Plus généralement, ce concept peut être étendu à l'utilisation par toute personne indépendamment de son sexe, son âge, sa situation ou son handicap ; on parle alors de conception universelle. D'un point de vue technique, cela peut aussi inclure la prise en compte d'une utilisation en mode « dégradé ».

analyseur syntaxique (*parser*) partie d'un **compilateur** chargée de l'analyse de la forme textuelle du code source et de la construction de l'**AST** correspondant.

API Interface de programmation d'application. C'est un ensemble de types de données (abstrait) et d'**opérations** associées, ou d'interfaces dans le monde objet. Elle définit la manière d'interagir avec un système ou une **bibliothèque** logicielle.

arbre syntaxique abstrait (AST) structure de donnée représentant un document ayant une grammaire formelle, par exemple le **code source** d'un programme, sous forme d'un arbre manipulable. Il est produit par l'**analyseur syntaxique**.

argument valeur passée à une **fonction**, **procédure** ou **méthode** lors de l'appel (généralement via la pile). Elle sera liée à la variable du **paramètre** correspondant dans le corps de la fonction. Cette correspondance peut être positionnelle ou par mot-clé. Selon le langage et la nature de l'argument, celui-ci peut être passé par valeur (copie) ou par **référence** (adresse).

assembleur langage de programmation très proche de la machine, et donc dépendant de l'architecture matérielle utilisée. Les instructions disponibles sont celles implémentées directement dans le processeur. Le code assembleur n'est généralement pas écrit par une personne, mais généré par un **compilateur** à partir de **code source** dans un langage plus haut niveau.

attribut (donnée membre, propriété, variable d'instance, champ) donnée (valeur) associée à chaque **instance** d'une classe ou d'un objet. Comme une variable, un attribut a un nom, et généralement un type associé. L'ensemble des attributs d'un objet constituent sa structure, leurs valeurs définissent son état.

autoboxing conversion automatique (implicite) d'un type primitif en type objet correspondant (et réciproquement), selon ce qui est nécessaire au contexte. Voir aussi **boxing**.

B

base64 codage d'information utilisant 64 caractères, utilisé notamment pour l'encodage des pièces jointes dans les messages électroniques. Il permet de transmettre des valeurs quelconques sur un canal n'autorisant qu'un nombre restreint de valeurs (p. ex. ASCII), comme les en-têtes **HTTP**. Voir JOSEFSSON, 2006.

bibliothèque ensemble de définitions de types, de structures de données et de traitements, généralement concernant un domaine ou une tâche précise, et conçue pour être réutilisable. On parle de bibliothèques standard pour celles fournies avec le langage, et de bibliothèques tierces pour celles développées indépendamment. Elles évitent de développer des fonctionnalités à partir de zéro dans chaque programme, mais créent une dépendance pour les bibliothèques tierces. Une bibliothèque définit souvent un **espace de noms** pour les éléments qu'elle contient. Contrairement au **framework**, une bibliothèque ne définit pas l'architecture de l'application, mais plutôt des « briques » élémentaires. Voir aussi **paquet & gestionnaire de dépendances**.

bookmarklet mini script (généralement en **ECMAScript**) stocké dans le **navigateur** sous forme de signet. La sélection du signet par l'utilisateur déclenche l'exécution du script sur la page en cours d'affichage dans le navigateur. Ceci permet d'ajouter simplement des fonctionnalités à une page quelconque de manière non-intrusive (rechercher le mot sélectionné dans un moteur de recherche, ajouter la page courante à un service d'annotation ou un réseau social, etc.)

bytecode (ou opcode) ensemble d'instructions de bas niveau, sorte d'assembleur, pour une **machine virtuelle**. Le bytecode est le résultat de la **compilation** d'un langage de plus haut niveau d'**abstraction**. Il est interprété par la machine virtuelle lors de l'exécution du programme.

C

cache système permettant la mémorisation d'une donnée afin de accélérer son utilisation ultérieure. La gestion de cache pour les données susceptibles de changer est un problème réputé difficile, notamment pour l'invalidation en cas d'expiration et la mise à jour, puisqu'on peut alors utiliser des données périmées.

cache (web) composant fournissant des fonctionnalités de **cache** pour les réponses aux requêtes **HTTP**. Les réponses aux requêtes sont mémorisées pour éviter d'envoyer plusieurs fois une même requête au serveur. Le cache peut être au sein d'un agent (p. ex. **navigateur**) ou d'un intermédiaire (*proxy...*). Certains intermédiaires sont dédiés à cette fonctionnalité.

callback **objet fonction** passé en paramètre à une **opération** afin d'être appelé quand cette dernière est terminée, généralement avec son résultat comme **argument**. L'utilisation la plus classique est dans la programmation asynchrone, où la fonction asynchrone, plutôt que de retourner son résultat, le passe en argument d'un appel à sa fonction *callback*. Concept proche des *handlers*.

classe définition abstraite un ensemble d'objets ayant des **propriétés** communes. Une classe peut être vue comme un modèle d'objets, servant à en dériver des **instances**. C'est elle qui en spécifie les **propriétés** caractéristiques. Elle définit donc aussi un **type**, à la fois son **interface** et son **implémentation**.

classe abstraite **classe** ne pouvant pas être instanciée directement, et devant donc être étendue. Elle contient des **méthodes abstraites** qui seront redéfinies dans les sous-classes, différant ainsi la définition d'une partie de son **implémentation**. Elle permet de définir un **type abstrait**. Voir aussi **extension**.

CLI interface en ligne de commande (CLI pour *command-line interface*). Interface utilisateur où l'interaction se fait par saisie de commande au clavier, généralement dans un shell. Les **REPL** peuvent aussi être considérées comme des CLI. Voir aussi **interface humain-machine**.

closure (ou fermeture) fonction capturant son contexte, c'est-à-dire les variables libres (non locales) dans le corps de la fonction qui existaient dans son environnement au moment de sa définition. Les closures sont souvent mises en œuvre avec les **lambda** définies dans une fonction génératrice. Une closure peut être vue comme le dual d'un **objet**.

CoD (*code on demand* pour «code à la demande») système permettant de spécialiser le comportement d'un logiciel en lui ajoutant des fonctionnalités, non prévues initialement. Contrairement aux extensions qui sont installées a priori dans le logiciel, la fonctionnalité est ici téléchargée et intégrée lorsqu'elle est nécessaire, à l'exécution.

code natif (ou binaire) instructions directement compréhensibles par le processeur, et donc dépendantes de l'architecture matérielle de l'ordinateur. On parle aussi de langage machine. Elles sont représentées sous forme de codes numériques, et stockées sous forme binaire (par opposition à une forme textuelle).

C'est l'équivalent binaire de l'**assembleur**. Il n'est pas écrit directement, mais généralement produit par un **compilateur** à partir de **code source**.

code source description d'un programme informatique écrit dans un langage de programmation. C'est du simple texte et n'importe quel éditeur peut servir à l'écrire, mais un **IDE** est souvent utilisé. Le code source sera ensuite converti par le **compilateur** en programme exécutable par l'ordinateur (**code natif**) ou en **bytecode**, ou directement exécuté par un **interpréteur**.

coercition fait de forcer le type d'une valeur vers un type différent (cible), sans en changer la représentation interne. Ce changement de type peut être explicite (*cast*) ou implicite lors de l'affectation ou utilisation comme argument où le type cible est requis. Un cas est l'utilisation de type incomparables mais ayant la même représentation interne (**int** et **char** en C par exemple). Ici, le changement concerne la manière d'interpréter la représentation en mémoire à l'exécution. L'autre cas courant est l'interprétation comme une instance d'un **super-type** (coercition ascendante ou *upcast*) ou d'un **sous-type** (coercition descendante ou *downcast*). Dans ce cas, le changement ne concerne que le système de vérification de type et de résolution statique au moment de la compilation. Voir aussi **transtypage**.

cohésion degré de liaison des composants internes d'un système unique (membres d'une classe, classes d'un **paquet**, etc.) On cherche en général à la maximiser. S'oppose au **couplage**.

compilateur programme effectuant la **compilation** du **code source**.

compilation transformation d'un artefact écrit dans un langage (généralement du **code source**) en un autre utilisant un langage ayant un plus bas niveau d'abstraction (généralement du **code natif**). Voir aussi **interprétation**.

complexité (algorithmique) analyse des ressources nécessaires à un algorithme proportionnellement à la taille des données à traiter. Cela peut être le temps, la quantité de mémoire, le nombre d'opérations, etc. Ne pas confondre avec la **complexité** d'un système.

complexité (système) fait d'être constitué d'un grand nombre d'éléments en interaction dont les comportements s'influencent mutuellement. Un système complexe a donc un comportement difficile à prévoir. S'oppose à **simple**, différent de compliqué ou difficile. Ne pas confondre avec la **complexité** algorithmique.

comportement façon dont un objet agit et réagit, du point de vue de ses changements d'état et de l'envoi de message. Généralement mis en œuvre par des **méthodes**.

composition création d'un **objet**, le composite, par assemblage d'autres objets, ses composants. Les **instances** des composants (ou leurs **références**) sont généralement stockées dans les **attributs** du composite. Les composants sont

totallement encapsulés dans le composite, et le plus souvent directement instanciés par celui-ci, les références n'étant normalement pas partagées. Avec la **délégation**, permet de simuler ou remplacer l'héritage.

constructeur **méthode** initialisant les nouvelles **instances** d'une classe. Généralement appelé automatiquement par le processus d'**instanciation**. Selon le langage, il se nomme comme la classe (en JAVA, C++) ou avec un nom prédéfini (**__init__** en PYTHON, **constructor** en ECMAScript). Voir aussi **destructeur**.

CORS (*Cross-origin resource sharing*) mécanisme de négociation entre un client et un serveur, permettant notamment d'effectuer des requêtes Ajax (XMLHttpRequests) vers un autre domaine que celui d'origine du script, normalement interdites. Voir KESTEREN, 2013.

couplage degré de liaison ou de dépendance entre éléments de composants différents. Représente donc la dépendance d'un élément vis-à-vis de l'extérieur, de son contexte. C'est l'inter dépendance des composants, leur degré d'interaction. On cherche en général à le minimiser, car c'est l'une des sources de la **complexité** des systèmes. S'oppose à la **cohésion**. On parle de couplage *abstrait* lorsque celui-ci se fait sur un élément abstrait, comme une **interface**.

crawler voir **robot**.

CSS (*Cascading Style Sheet*) langage permettant la mise en forme de données formatées en **XML**, plus particulièrement les documents **HTML**.

D

délégation mécanisme par lequel un objet fait suivre, ou délègue, un traitement à un autre objet. Ce peut être une alternative à l'héritage, ou même à l'organisation en **classes** dans les langages à prototype (comme SELF ou ECMAScript). La délégation permet de partager le comportement défini dans un objet sans introduire de sous-type ni d'héritage. On parle de réutilisation « boîte noire », car le fonctionnement interne de l'objet n'est jamais exposé (voir aussi **composition**). Elle est plus flexible et plus dynamique que l'héritage. Elle peut être simulée manuellement, ou gérée nativement par le langage. Voir LIEBERMAN, 1986.

DOM (*Document Object Model*) représentation d'un document **HTML** (ou tout document **XML**), sous la forme d'une structure de donnée manipulable par un programme. Il prend la forme d'un arbre, où chaque nœud est un objet fournissant une interface homogène permettant de le manipuler. C'est un standard défini par le **WHATWG**. Voir **DOM 2020**.

DTD (*Document Type Definition*) définition de la grammaire formelle utilisée par un langage basé sur **SGML** ou **XML**, et donc de sa syntaxe abstraite.

dynamique se dit d'une propriété ou valeur qui n'est connue, fixée ou définie qu'au moment de l'exécution, ou qui peut changer au cours de l'exécution du programme (*run time*). S'oppose à **statique**

E

ECMA International organisation de standardisation en informatique, normalisant notamment ECMAScript (ECMA-262), CLI (ECMA-335) et C# (ECMA-334), Eiffel (ECMA-367).

ECMAScript langage dynamique utilisé notamment dans les navigateurs Web. Voir *ECMAScript Language Specification 2020*.

effet de bord modification de l'environnement par l'exécution d'une **opération**. Une **opération** agit par effet de bord notamment lorsqu'elle modifie une variable non locale, une donnée extérieure passée par **référence**, ou effectue une entrée/sortie. Voir aussi **transparence référentielle & procédure**.

encapsulation regroupement dans une même entité des données sémantiquement liées, et des traitements qui s'y rapportent. Liée au **masquage d'information**.

espace de noms contexte pour les éléments ou **identifiants** qu'il contient. Il permet notamment d'éviter les collisions dans les noms de ceux-ci et les ambiguïtés qui pourrait en découler. Les espaces de noms peuvent parfois être utilisés de manière récursive, c'est-à-dire qu'un espace peut lui-même être contenu dans un espace de noms.

exception erreur imprévue qui surgit dans un programme. Une exception est *levée* au point d'erreur et se propage dans la pile d'appel pour être traitée à un autre endroit du programme, interrompant le flot normal d'exécution. C'est généralement un objet à part entière.

expression élément du langage (ensemble d'opération, valeurs, appels de fonctions) ayant une valeur. Une expression peut donc être affectée à une variable, passée en **argument** d'une fonction, etc. S'oppose à **instruction**.

extension principe permettant de définir un nouveau type en le faisant dériver d'un **super-type**. Le nouveau type possède donc implicitement toutes les **propriétés** de son **super-type**, ainsi que certaines propriétés propres. L'extension peut-être facilitée par l'**héritage** de, ou la **délégation** vers, des types (partiellement) concrets.

F

fermeture voir *closure*.

fonction **opération** retournant une valeur comme résultat. C'est donc une **expression**. Idéalement, elle ne doit pas avoir d'**effet de bord**, afin d'être référentiellement transparent (cf. **transparence référentielle**). On parle alors de *fonction pure*. Voir aussi **procédure**.

fonction anonyme *fonction* n'étant pas associée à un *identifiant*. Elle est la plupart du temps utilisée comme *argument* d'une *fonction d'ordre supérieur*. Elle représente l'équivalent pour les *type fonctionnel* des notations littérales pour les types de données. *Voir aussi objet fonction*.

fonction d'ordre supérieur *fonction* manipulant des fonctions. Cela peut être une fonction dont le comportement est spécifié par un traitement passé en paramètre sous la forme d'un *objet fonction* ou une fonction créant des fonctions (souvent des *closures*).

framework ensemble de classes, structures de données, et fonctions, définissant une architecture réutilisable pour une famille de logiciel. Contrairement aux *bibliothèques*, il définit le comportement général, abstrait, de l'architecture. Seul le comportement spécifique à une application particulière devra être défini au cas par cas. Cette spécialisation peut se faire par extension de classes abstraites du framework, par composition et injection de dépendances à la configuration, etc.

G

généralisation TODO S'oppose à la *spécialisation*.

GUI interface graphique (GUI pour *graphical user interface*). Interface utilisateur visuelle interactive, constituée de boutons, champs de saisie, icônes et représentations graphiques riches, éventuellement agencées dans un système de fenêtres d'affichages indépendantes. *Voir aussi interface humain-machine*.

H

handler en programmation événementielle, *objet fonction* associé au déclenchement d'un événement. Cette fonction sera appelée par la boucle événementielle lorsque l'événement associé se produit, avec celui-ci comme *argument*.

héritage accès par les *instance* de la *sous-classe* aux *propriétés* définies dans sa *super-classe*. Permet donc de définir une nouvelle classe en dérivant une ou plusieurs *super-classe*. L'héritage facilite l'*extension* et le *sous-typage* en faisant de la réutilisation. On parle de réutilisation « boîte blanche », car la sous-classe est liée au fonctionnement interne de la *super-classe*. La *spécialisation* de la sous-classe peut nécessiter de redéfinir certaines méthodes héritées (voir *redéfinition*). L'héritage est transitif. Alternative à la *délégation*.

HOF voir *fonction d'ordre supérieur*

HTML (*Hypertext Markup Language*) langage de structuration de documents utilisé pour le *Web*. Il permet de représenter le contenu d'un document de manière sémantique, indépendamment de la manière dont il est présenté.

HTTP (*Hypertext Transfer Protocol*) Protocole de communication réseau (niveau application) conçu pour le *Web*.

I

IDE (*Integrated Development Environment*) environnement de développement, intégrant divers outils d'aide à la programmation et au génie logiciel.

identifiant nom symbolique associé à une entité du programme (fonction, variable, type) pour le désigner et le manipulé. Il est généralement associé à une valeur (voir *liaison*).

IHM voir *interface humain-machine*

implémentation code effectif d'une structure ou d'une *opération*. L'implémentation doit au moins contenir le code associé aux méthodes spécifiées dans l'interface associée au type concret. S'oppose à l'*interface*.

instance l'instance d'une *classe* est une valeur dont le type est la classe. C'est un des éléments de l'ensemble décrit par la classe. Elle contient les valeurs associées aux *attributs* définis dans la classe correspondante, qui représentent son état, et en expose les *méthodes*.

instanciation processus de création de nouveaux objets (généralement *instances* d'une *classe*). Elle se décompose en deux parties : l'allocation mémoire et l'initialisation. L'allocation se fait via l'opérateur *new* (JAVA, ECMAScript) ou par appel d'une *méthode de classe* (*__new__* en PYTHON, *new* en SMALLTALK). L'initialisation se fait via le *constructeur*.

instruction élément du langage (opérations, structures de contrôle, appels de fonctions) ayant un effet sur l'état du système. S'oppose à *expression*.

interface (concept) L'interface d'une *classe*, d'un *type* ou d'un module est l'ensemble des *propriétés* mises à disposition. Elle définit la manière dont on peut interagir avec les *instances* de ce type. S'oppose à l'*implémentation*.

interface (type) élément du langage définissant un type purement abstrait, sous la forme d'un ensemble de *signatures* de *méthodes*. Une interface ne peut pas contenir d'*attributs*, ceux-ci étant du domaine de l'*implémentation*.

interface fonctionnelle *interface* ne spécifiant qu'une seule méthode, et représentant donc un *objet fonction*. Dans les langages disposant de *lambda*, celles-ci peuvent être utilisées comme instances d'interfaces fonctionnelles. Équivalent des *type fonctionnels*. Voir aussi *SAM*.

interface humain-machine système d'interaction entre un utilisateur ou utilisatrice et un ordinateur ou logiciel. Cela peut être un périphérique matériel spécifique ou désigner la partie du logiciel avec lequel l'utilisateur ou utilisatrice interagit. On parle alors d'interface graphique (*GUI*), textuelle (*TUI*), en ligne de commande (*CLI*), etc.

interprétation exécution directe d'un langage plus haut niveau que du *code natif* (*code source*, *AST* ou représentation intermédiaire, *bytecode*) sans passer par une phase de *compilation*. On parle aussi d'évaluation *dynamique*. Voir aussi *interpréteur*.

interprète voir **interpréteur**

interpréteur (parfois **interprète**) programme effectuant l'**interprétation** d'un langage de haut niveau, par opposition à un **compilateur**.

J

JSON (*Javascript Object Notation*) format d'échange et de stockage de données génériques sous forme textuelle. Il est dérivé de la notation permettant de spécifier des objets de manière littérale en **ECMAScript**. Il a un usage similaire à **XML**, auquel il est souvent préféré. Comme lui, il ne définit qu'une syntaxe concrète, sans schéma particulier. C'est un standard défini dans *The JSON data interchange syntax 2017* et BRAY, 2017.

L

lambda voir **fonction anonyme**

liaison (*binding*) association d'une valeur (ou d'une expression) à un **identifiant**, comme une variable et sa valeur, ou un appel de méthode au code effectif de l'**opération**. Les liaisons **statiques** (ou *immédiates*) sont résolues à la **compilation** ; les liaisons **dynamiques** (ou *différées, tardives*) sont effectuées à l'exécution (*late binding*).

M

machine virtuelle environnement d'exécution de *bytecode* jouant le rôle d'**abstraction** de l'infrastructure matérielle réelle.

masquage d'information le fait de ne rendre accessible que le nécessaire aux utilisateurs d'un composant, en masquant les détails d'**implémentation** ou les caractéristiques non essentielles (état, **implémentation** des méthodes, ...). Les éléments masqués ne sont pas accessibles à l'extérieur de l'objet. La représentation interne de l'objet (son état) ne peut donc être modifiée que par l'intermédiaire des méthodes exposées, et donc de manière contrôlée. Cela permet de diminuer le **couplage** entre composant. Lié à l'**encapsulation**. Voir notamment PARNAS, 1972.

membre voir **propriété**

méta-classe **classe** d'un objet classe. Dans certains langages, les classes sont elles-mêmes des **objets**, et peuvent être manipulées comme des valeurs (passage en **argument**, envoi de message, etc.). Elles sont donc les **instances** d'une classe particulière : la méta-classe. En JAVA, c'est la classe **Class** qui fixée et finale. En PYTHON, c'est **type** qui peut être étendue et changée pour chaque classe. En SMALLTALK, chaque classe a sa propre méta-classe (elle-même sous-classe de la classe **Class**). Étendre et redéfinir la méta-classe permet de spécialiser le comportement des classes elles-mêmes. Ainsi, les **propriétés** statiques ou de classes peuvent être vue comme des **membres** d'instances de la méta-classe (c'est exactement le cas en SMALLTALK). Voir aussi **méta-programmation**.

méthode **procédure** ou **fonction** associée à un objet (généralement via sa classe) et invoquée par envoi de message. La **résolution** est **dynamique** en fonction de l'objet **receveur** du message. Celui-ci sera le premier **argument** (souvent implicite) de l'appel de la méthode.

méthode abstraite **méthode** n'ayant pas de code associé, pas d'**implémentation**. Seule sa **signature** est spécifiée. Une méthode abstraite ne peut exister que dans une **classe abstraite**. Les **sous-classes** de la classe abstraite devront définir une **implémentation** pour cette méthode, ou être elles-mêmes abstraites. On parle aussi de méthode virtuelle pure, ou différée (voir **redéfinition**).

méthode de classe méthode dont le **receveur** est la **classe** elle-même, vue comme une instance de la **méta-classe**, et non une instance de la classe. Cette notion n'existe pas en JAVA. Ne pas confondre avec les **méthodes statiques**.

méthode statique méthode qui peut être appelée hors d'un contexte objet ; elle n'a donc pas de **receveur**, et la **liaison** est par conséquent **statique**. Elle est cependant définie dans l'**espace de noms** de la classe. Pour définir une méthode statique beaucoup de langages utilisent le mot-clé **static**.

N

navigateur logiciel permettant de se déplacer sur le **Web** de manière interactive (par opposition par exemple à un **robot**). C'est un client générique (par opposition à une application spécifique), qui peut donc servir à consulter n'importe quel site. Il contient généralement un client **HTTP**, un moteur de rendu **HTML** et **CSS**, ainsi qu'une **machine virtuelle** pour **ECMAScript**.

O

objet entité autonome regroupant un état (des données) et les traitements permettant de manipuler cet état. Peut être vu comme l'**encapsulation** d'une structure de donnée et des **opérations** qui s'y rapportent.

objet fonction valeur de premier ordre représentant la réification d'un traitement. Dans les langages fonctionnel, les fonctions elles-mêmes sont des valeurs de premier ordre. Les fonctions anonymes (**lambda**) sont aussi des objets fonction. Dans les langages objet, on utilise généralement des instances d'**interfaces fonctionnelles** ou de **SAM**.

opaque se dit d'un élément ou d'un composant que l'on utilise sans en connaître le fonctionnement ou la sémantique interne. On parle d'utilisation en « boîte noire ». Cet aspect dépend du point de vue, un élément opaque pour un acteur (utilisateur) peut ne pas l'être pour un autre (celui qui le définit). Différent d'obscur, incompréhensible. S'oppose à **transparent**.

opération **abstraction** représentant un traitement réutilisable. *Voir aussi* **fonction**, **procédure** & **méthode**.

P

paquet (package) regroupement de plusieurs classes et interfaces, ou structures de données et **opérations** associées. Ce regroupement a un but sémantique et organisationnel. Il définit également un **espace de noms**. En JAVA, il est déclaré avec le mot-clé **package**.

paramètre variable libre dans le corps d'une **fonction**, **procédure** ou **méthode**, dont la valeur sera spécifiée à l'appel par passage d'**argument**. De manière plus générale, c'est un élément d'information utilisé dans la description d'une entité (type, structure, traitement) qui sera spécifié au moment de l'évaluation.

pointeur valeur représentant l'adresse mémoire d'une valeur et donc directement manipulable.

polymorphisme (lit. plusieurs formes) possibilité de voir un objet de plusieurs façons, comme appartenant à plusieurs **types** à la fois, ou pour une **opération** de s'appliquer à plusieurs types. Il existe différentes formes de polymorphisme (voir notamment **transtypage**, **redéfinition**, **surcharge**, **type générique**, **sous-typage**).

procédure **opération** fonctionnant par **effet de bord**, et ne retournant pas de valeur. Un appel de procédure est donc une **instruction** et non une **expression**. Voir aussi **fonction**.

propriété donnée (**attribut**) ou comportement (**méthodes**) associée à un objet. Les propriétés sont généralement définies dans la **classe** correspondante.

protocole description formelle de l'interaction entre différents composants, sous la forme des séquences d'**opérations** autorisées.

PWA (*progressive web application*) Type de **SPA** conçue pour pouvoir être « installée » sur un système, c'est-à-dire fournir une icône de raccourci (notamment sur périphérique mobile) similaire aux applications locales, éviter le téléchargement systématique du code et même dans une certaine mesure d'être utilisée hors connexion. L'application s'exécute cependant toujours dans un navigateur, certains utilisant une fenêtre spécifique et isolée.

R

receveur (cible) objet recevant le message. C'est celui sur lequel on appelle la **méthode**, dont il sera le premier **argument** (souvent implicite). Dans le corps de la méthode, il est généralement désigné par **this** ou **self**.

redéfinition (*overriding*) **implémentation** alternative d'une méthode déjà définie dans une **super-classe**, et donc héritée, spécifiée dans la **sous-classe**. La redéfinition masque la méthode héritée. Cela permet de fournir une **implémentation** spécifique au **sous-type**, et permet donc une forme de **polymorphisme ad hoc**. La **résolution** se fait en fonction du **receveur** et est **dynamique**.

référence adresse de l'espace mémoire où est stockée une valeur (généralement dans le tas). Contrairement à un **pointeur**, une référence n'est pas manipulable

directement. Une variable contenant la référence devient un alias pour la valeur, cette dernière n'est pas copiée. En JAVA, comme dans beaucoup de langages haut niveau où la mémoire est gérée, les types complexes ne sont accessibles que par leur références.

REPL environnement de commande interactif (REPL pour *read-eval-print loop*), où l'on interagit avec un programme en tapant des commandes directement dans un **interpréteur** interactif. Le résultat de l'exécution de la commande est affiché immédiatement. La plupart du temps, il s'agit d'un mode textuel dans un terminal, mais il existe des environnements plus riches (p. ex. la console des navigateurs). *Voir aussi* **interface humain-machine**.

résolution (de méthode) processus permettant d'associer un appel symbolique particulier (envoi de message) à la méthode concrète à exécuter. La recherche de l'**implémentation** dépend de la **signature** de la méthode, de ses **arguments** et de l'objet **receveur**. Elle peut être **statique** ou **dynamique**. La recherche peut suivre l'arbre d'**héritage** ou le graphe de **délégation** si la méthode n'est pas définie dans le receveur. C'est un des éléments permettant le **polymorphisme ad hoc**, notamment la **surcharge** et la **redéfinition**. Le terme s'utilise aussi hors du contexte du paradigme objet, lors de la **surcharge** notamment. *Voir aussi* **liaison**.

REST *representational state transfer* ensemble de contraintes architecturales permettant de définir des systèmes d'informations interactifs à grande échelle, tels que le **Web**, notamment par l'utilisation de l'hypermédia. Voir R. T. FIELDING, 2000.

RIA *rich internet application*. Application web riche, s'exécutant directement dans un navigateur, offrant des fonctionnalités similaires à une application de bureau « classique », mais sans nécessiter d'installation autre que le navigateur (et éventuellement des extensions à celui-ci selon la technologie utilisée), le programme étant téléchargé à chaque exécution.

robot logiciel (agent) navigant sur le web de manière autonome, en suivant les hyperliens présents dans les pages. Il est généralement utilisé pour extraire de l'information des pages visités (*web scraping*), par exemple pour créer un index de recherche. On parle aussi de *crawler*.

S

SAM (*single abstract method*) **classe abstraite** ayant une seule méthode abstraite. Ces classes sont la version objet des fonctions comme valeur de premier ordre de la programmation fonctionnelle, la méthode abstraite représentant l'opération à effectuer. Équivalent aux **interfaces fonctionnelles**.

script tiers code client indépendant, généralement en **ECMAScript**, s'exécutant dans le site web mais sert à partir d'une adresse externe à celui-ci, tels que

les scripts d'analyse de trafic, les boutons pour réseaux sociaux, les cartes embarquées, etc.

SGML (*Standard Generalized Markup Language*) méta-langage servant de support pour la création de langages à balises. Une instance de langage basée sur SGML doit définir l'ensemble des balises valides et leur imbrication (la grammaire) dans une **DTD** associée.

signature (de fonction ou méthode) ensemble des types des paramètres et de retour d'une **opération** (y compris les erreurs et **exceptions**) définissant ainsi son propre type. On parle aussi de **type fonctionnel**. C'est notamment ce qui est utilisé dans la **résolution**. Le nom de l'opération est parfois inclus dans la signature.

simple (système) S'oppose à **complexité**, différent de facile.

SOA architecture orientée service (*Standard Generalized Markup Language*). Conception architecturale modulaire, où chaque module est implémenté sous la forme d'un service autonome. L'application globale est donc construite par composition de ces services élémentaire (on parle aussi d'orchestration).

sous-classe **classe** définissant un **sous-type** pour autre classe, sa **super-classe**, généralement par le biais de l'héritage de ses **propriétés**. On parle aussi de classe fille, classe enfant, ou classe dérivée. La sous-classe peut ajouter ses propres propriétés à celles de sa super-classe; on parle ainsi d'**extension**.

sous-typage relation (*is a*) définissant un **sous-type**. Le sous-typage peut être structurel ou nominal. Il permet le **polymorphisme** d'inclusion. *Voir aussi substituabilité & spécialisation.*

sous-type **type** défini tel que les **instances** d'un sous-type sont des formes spécialisées du **super-type** (relation de **sous-typage**). D'un point de vue ensembliste, l'ensemble des instances du sous-type est inclus dans l'ensemble des instances du super-type. Elles peuvent donc être utilisées à la place d'instances du super-type.

SPA (*single page application*) application riche, construite avec les technologies du **Web** (**HTML**, **CSS**, **ECMAScript**) et utilisant un **navigateur** comme environnement d'exécution. Le nom vient du fait que contrairement aux interfaces hypermédia du web classique, l'application s'exécute sans navigation du point de vue du navigateur, toute l'interaction étant gérée par **ECMAScript**, et donc dans une seule page. Architecturalement parlant, ce ne sont donc pas des applications hypermédia, mais des **GUI** «classique», dont l'environnement est le navigateur et dont le code est téléchargé à chaque exécution. *Voir aussi RIA & PWA.*

spécialisation **TODO** S'oppose à la **généralisation**

statique se dit d'une propriété ou valeur qui est connue, définie ou fixée au moment de la **compilation** (*compile time*), et donc inchangée au cours de l'exécution du programme. S'oppose à **dynamique**.

sucre syntaxique élément syntaxique d'un langage de programmation facilitant l'écriture mais n'apportant pas de nouvelle fonctionnalité intrinsèque au langage. Le sucre syntaxique est souvent ajouté pour faciliter l'utilisation de constructions fréquentes, en fournissant une écriture plus succincte (p. ex. boucle **foreach** sur un itérateur, valeur par défaut de paramètres, fonctions variadiques, etc.) La nouvelle forme syntaxique est généralement transformée en code du langage dans les étapes préliminaires de la **compilation**. Les langages possédant un système de macros permettent, dans une certaine mesure, à l'utilisateur de définir de telles constructions.

super-classe classe servant de **super-type** à une ou plusieurs autres classe, ses **sous-classes**. On parle aussi de classe mère ou de classe parente.

super-type voir **sous-type**.

surcharge (*overloading*) fait d'associer plusieurs **opérations** de **type fonctionnel** différents à un même **identifiant**. La surcharge d'une fonction, d'un opérateur, ou d'une méthode consiste à déclarer une fonction plusieurs fois, avec des paramètres différents, dans le même **espace de noms** (module ou classe), ou de les importer dans un espace de nom commun si la **résolution** est **dynamique**. C'est un des éléments du **polymorphisme ad hoc**. La **redéfinition** peut être considérée comme un type de surcharge (au sens large), mais le terme désigne plutôt la surcharge *paramétrée*, où la **résolution** se base uniquement sur la **signature** de la fonction. Dans beaucoup de langages, la résolution de la surcharge est **statique**.

T

TCP (*Transmission Control Protocol*) protocole de transport en mode connecté, utilisant la couche IP (couche réseau)

transparence référentielle fait pour une **expression** de pouvoir être remplacée par sa valeur sans altérer le comportement du programme. Cette propriété permet au **compilateur** ou à l'**interpréteur** de faire certaines optimisations. Elle facilite aussi le raisonnement sur l'exactitude du programme. La transparence référentielle est garantie dans les langages fonctionnels purs. Elle est en effet incompatible avec les **effets de bord** visibles.

transparent se dit d'un composant invisible, dont la présence n'est pas perceptible à l'utilisation. Différent de clair, évident. S'oppose à **opaque**.

transtypage conversion d'une valeur d'un certain type en une valeur équivalente d'un autre type (cible). Cette conversion est généralement automatique et implicite, et implique un changement dans la représentation interne de la valeur. Par exemple, la conversion d'une valeur de type entier **int** en **long**, ou même en **float**, lors de son affectation à une variable déclarée du type cible, ou lors d'une utilisation comme argument dans un appel de fonction acceptant le type cible.

Il permet une forme de **polymorphisme**. La **coercition** est parfois considérée comme du transtypage, bien qu'elle n'en soit pas au sens strict.

TUI interface textuelle (TUI pour *text-based user interface*), ou en mode texte. Interface utilisateur visuelle interactive, affichée uniquement avec du texte et des éléments graphiques simples, généralement dans un terminal. L'interaction se fait généralement principalement au clavier. Voir aussi **interface humain-machine**.

type ensemble de valeurs partageant une structure et un **comportement** commun.

type abstrait formalisation théorique d'un type de donnée, décrit par les valeurs possibles qu'il peut prendre et par son comportement, et donc les **opérations** pouvant être y appliquées, ainsi que les pré et post conditions et invariants qui doivent être respectés (son **interface**). C'est une description abstraite, indépendante d'une quelconque **implémentation**. Voir notamment LISKOV et ZILLES, 1974. Dans les langages objets, mis en œuvre par une **classe abstraite** ou une **interface**.

type fonctionnel **type** d'un **objet fonction**, représenté par la **signature** des fonctions instances, dans les langages fonctionnels, où les fonctions sont des valeurs de premier ordre. Souvent présent dans la signature des **fonction d'ordre supérieur**.

type générique type complexe dont le type d'un ou plusieurs constituant n'est pas fixé *a priori*, mais défini à l'**instanciation** ou à l'**extension**. Le type est donc un paramètre. Permet un type de **polymorphisme** universel.

U

URI *Uniform Resource Identifier*. Voir BERNERS-LEE, R. T. FIELDING et MASINTER, 2005

URL (*Uniform Resource Locator*) système de nommage unique d'une ressource sur le Web, dépendant du protocole utilisé. C'est une référence de la ressource qui est déréférençable. Une URL est **opaque** pour le client. Voir BERNERS-LEE, MASINTER et MCCAILL, 1994.

UUID (*Universal Unique Identifier*) identifiant unique global sur 128 bits pouvant être généré sans coordination centrale (voir notamment LEACH, MEALLING et SALZ, 2005)

W

W3C (*World Wide Web Consortium*) organisme international chargé de développer et standardiser les technologies relatives au **Web** (<http://www.w3.org/>)

Web voir **WWW**

WHATWG (*Web Hypertext Application Technology Working Group*) groupe de travail regroupant les principaux éditeurs de **navigateurs** Web. Il a pour but la spécification et le développement des technologies en lien avec le développement d'applications basées sur le **Web**.

WWW (*World Wide Web* ou simplement *le Web*) système d'information global. C'est une interconnexion de ressources (pages) via des hyperliens, identifiées par leur **URL** dans le protocole **HTTP**.

X

XML (*eXtensible Markup Language*) méta-langage, dérivé de **SGML**. Il est plus strict que ce dernier, moins ambigu, et est donc plus facile à analyser.

Bibliographie

- BARTH, A. (déc. 2011a). *The Web Origin Concept*. RFC 6454. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc6454>.
- BERNERS-LEE, T. J. (1992). *Style Guide for online hypertext*. URL : <http://www.w3.org/Provider/Style/>.
- (1996). *Universal Resource Identifiers : Axioms of Web Architecture*. URL : <http://www.w3.org/DesignIssues/Axioms>.
- (1998). *Cool URIs don't change*. URL : <http://www.w3.org/Provider/Style/URI>.
- (2000). *Weaving the Web*. 1^{re} éd. Harper Business, p. 256. ISBN : 978-0062515872.
- (2017). *Design Issues*. URL : <http://www.w3.org/DesignIssues/>.
- BERNERS-LEE, T. J., R. T. FIELDING et L. M. MASINTER (jan. 2005). *Uniform Resource Identifier (URI) : Generic Syntax*. RFC 3986. Network Working Group. URL : <http://tools.ietf.org/html/rfc3986>.
- BERNERS-LEE, T. J., L. M. MASINTER et M. MCCAHILL (déc. 1994). *Uniform Resource Locators*. RFC 1738. Network Working Group. URL : <http://tools.ietf.org/html/rfc1738>.
- The JavaScript Object Notation (JSON) Data Interchange Format* (déc. 2017). RFC 8269. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc8259>.
- BROWN, A. et G. CLEMM (avr. 2010). *Link Relation Types for Simple Version Navigation between Web Resources*. RFC 5829. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc5829>.
- A Convention for HTTP Access to JSON-Based Resources* (fév. 2012). Internet-Draft. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/draft-pbryan-http-json-resource-02>.
- JavaScript Object Notation (JSON) Patch* (avr. 2013). RFC 6902. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc6902>.

- CLARK, J. (déc. 1997). *Comparison of SGML and XML*. Note NOTE-sgml-xml-971215. The World Wide Web Consortium. URL : <http://www.w3.org/TR/NOTE-sgml-xml>.
- Internet Web Replication and Caching Taxonomy* (jan. 2001). RFC 3040, p. 32. URL : <http://tools.ietf.org/html/rfc3040>.
- CROCKFORD, D. (juill. 2006). *The application/json Media Type for JavaScript Object Notation (JSON)*. RFC 4627. Network Working Group. URL : <http://tools.ietf.org/html/rfc4627>.
- DOM (2020). Recommendation. WHATWG. URL : <https://dom.spec.whatwg.org/>.
- DUSSEAULT, L. et J. SNELL (mars 2010). *PATCH Method for HTTP*. RFC 5789. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc5789>.
- ECMAScript Language Specification* (juin 2020). ECMA 262. Ecma International. URL : <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>.
- The JSON data interchange syntax* (déc. 2017). ECMA 404. ECMA. URL : <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>.
- FETTE, I. et A. MELNIKOV (déc. 2011). *The WebSocket Protocol*. RFC 6455. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc6455>.
- Hypertext Transfer Protocol (HTTP/1.1) : Semantics and Content* (juin 2014). RFC 7231. Internet Engineering Task Force (IETF). URL : <https://tools.ietf.org/html/rfc7231>.
- FIELDING, R. T. (2000). « Architectural Styles and the Design of Network-based Software Architectures. » Thèse de doct. Irvine : University of California. URL : <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- HOLTMAN, K. (avr. 1998). *The Safe Response Header Field*. RFC 2310. Network Working Group. URL : <http://tools.ietf.org/html/rfc2310>.
- IANA, éd. (s. d.[a]). *Hypertext Transfer Protocol (HTTP) Authentication Scheme Registry*. URL : <http://www.iana.org/assignments/http-authschemes/>.
- éd. (s. d.[b]). *Message Headers*. URL : <http://www.iana.org/assignments/message-headers>.
- MIME Media Types* (s. d.). URL : <http://www.iana.org/assignments/media-types/>.
- IANA, éd. (s. d.[c]). *Hypertext Transfer Protocol (HTTP) Status Code Registry*. URL : <http://www.iana.org/assignments/http-status-codes/>.
- Architecture of the World Wide Web* (déc. 2004). Recommendation. The World Wide Web Consortium. URL : <http://www.w3.org/TR/webarch/>.
- JOSEFSSON, S. (oct. 2006). *The Base16, Base32, and Base64 Data Encodings*. RFC 4648. Internet Engineering Task Force (IETF). URL : <https://tools.ietf.org/html/rfc4648>.

- Cross-Origin Resource Sharing* (déc. 2013). Proposed Recommendation. The World Wide Web Consortium. URL : <http://www.w3.org/TR/cors/>.
- LEACH, P., M. MEALLING et R. SALZ (juill. 2005). *A Universally Unique Identifier (UUID) URN Namespace*. RFC 4122. Internet Engineering Task Force (IETF). URL : <https://tools.ietf.org/html/rfc4122>.
- LIEBERMAN, H. (1986). « Using Prototypical Objects to Implement Shared Behavior in Object-Oriented Systems ». In : *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications*. OOPSLA '86. Portland, Oregon, USA : Association for Computing Machinery, p. 214-223. ISBN : 0897912047. DOI : [10.1145/28697.28718](https://doi.org/10.1145/28697.28718).
- LISKOV, B. et S. ZILLES (1974). « Programming with Abstract Data Types ». In : *Proceedings of the ACM SIGPLAN Symposium on Very High Level Languages*. Association for Computing Machinery, p. 50-59. ISBN : 9781450378840. DOI : [10.1145/800233.807045](https://doi.org/10.1145/800233.807045).
- MDN (s. d.). *Mozilla Developer Network*. URL : <https://developer.mozilla.org/tr/>.
- NOTTINGHAM, M. (mai 2010a). *HTTP Cache-Control Extensions for Stale Content*. RFC 5861. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc5861>.
- (oct. 2010b). *Web Linking*. RFC 5988. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc5988>.
- URI Design and Ownership* (juin 2020). RFC 8820. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc8820>.
- NOTTINGHAM, M. et E. HAMMER-LAHAV (avr. 2010). *Defining Well-Known Uniform Resource Identifiers (URIs)*. RFC 5785. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc5785>.
- PARNAS, D. L. (1972). « On the Criteria to Be Used in Decomposing Systems into Modules ». In : *Communication of the ACM* 15.12, p. 1053-1058. ISSN : 0001-0782. DOI : [10.1145/361598.361623](https://doi.org/10.1145/361598.361623).
- Guidelines for Web Content Transformation Proxies* (oct. 2010). Working Group Note. The World Wide Web Consortium. URL : <http://www.w3.org/TR/ct-guidelines/>.
- RAUSCHMAYER, A. (2018). *Exploring ES6. Upgrade to the next version of JavaScript*. LeanPub, p. 641. URL : <https://exploringjs.com/es6/index.html>.
- MASINTER, L. M. (avr. 1998). *Hyper Text Coffee Pot Control Protocol (HTCPCP/1.0)*. RFC 2324. Network Working Group. URL : <http://tools.ietf.org/html/rfc2324>.
- RESCORLA, E. (mai 2000). *HTTP Over TLS*. RFC 2818. Network Working Group. URL : <http://tools.ietf.org/html/rfc2818>.
- The Atom Syndication Format* (déc. 2005). RFC 4287. Network Working Group. URL : <http://tools.ietf.org/html/rfc4287>.
- The Atom Publishing Protocol* (oct. 2007). RFC 5023. Network Working Group. URL : <http://tools.ietf.org/html/rfc5023>.

- BARTH, A. (avr. 2011b). *HTTP State Management Mechanism*. RFC 6265. Internet Engineering Task Force (IETF). URL : <http://tools.ietf.org/html/rfc6265>.
- SNOOK, J. (2012). *Scalable and Modular Architecture for CSS*. 2^e éd., p. 110. ISBN : 978-0-9856321-0-6. URL : <http://smacss.com/book>.
- SYDIK, J. J. (2007). *Design Accessible Web Sites*. The Pragmatic Programmers, p. 300.
- W3C (s. d.). *The World Wide Web Consortium*. URL : <http://www.w3.org/>.
- YERGEAU, F. (nov. 2003). *UTF-8, a transformation format of ISO 10646*. RFC 3629. The Internet Society, p. 14. URL : <http://tools.ietf.org/search/rfc3629>.

version du 17 septembre 2025